



PDF Download
3460319.3464818.pdf
18 March 2026
Total Citations: 12
Total Downloads: 960



Published: 11 July 2021

Citation in BibTeX format

ISSTA '21: 30th ACM SIGSOFT
International Symposium on Software
Testing and Analysis
July 11 - 17, 2021
Virtual, Denmark

Conference Sponsors:
SIGSOFT

 Latest updates: <https://dl.acm.org/doi/10.1145/3460319.3464818>

RESEARCH-ARTICLE

SAND: a static analysis approach for detecting SQL antipatterns

YINGJUN LYU, Amazon.com, Inc., Seattle, WA, United States

SASHA VOLOKH, University of Southern California, Los Angeles, CA, United States

WILLIAM G. J. HALFOND, University of Southern California, Los Angeles, CA, United States

OMER TRIPP, Amazon.com, Inc., Seattle, WA, United States

Open Access Support provided by:

University of Southern California

Amazon.com, Inc.

SAND: A Static Analysis Approach for Detecting SQL Antipatterns

Yingjun Lyu*

Amazon
USA

William G. J. Halfond

University of Southern California
USA

Sasha Volokh

University of Southern California
USA

Omer Tripp

Amazon
USA

ABSTRACT

Local databases underpin important features in many mobile applications, such as responsiveness in the face of poor connectivity. However, failure to use such databases correctly can lead to high resource consumption or even security vulnerabilities.

We present SAND, an extensible static analysis approach that checks for misuse of local databases, also known as SQL antipatterns, in mobile apps. SAND features novel abstractions for common forms of application/database interactions, which enables concise and precise specification of the antipatterns that SAND checks for.

To validate the efficacy of SAND, we have experimented with a diverse suite of 1,000 Android apps. We show that the abstractions that power SAND allow concise specification of all the known antipatterns from the literature (12-74 LOC), and that the antipatterns are modeled accurately (99.4-100% precision). As for performance, SAND requires on average 41 seconds to complete a scan on a mobile app.

CCS CONCEPTS

• **Software and its engineering** → **Software defect analysis**; **Software defect analysis**; • **Security and privacy** → *Software and application security*;

KEYWORDS

Mobile applications; database; performance; security.

ACM Reference Format:

Yingjun Lyu, Sasha Volokh, William G. J. Halfond, and Omer Tripp. 2021. SAND: A Static Analysis Approach for Detecting SQL Antipatterns. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '21)*, July 11–17, 2021, Virtual, Denmark. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3460319.3464818>

*This research was done while Yingjun Lyu worked at the University of Southern California.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ISSTA '21, July 11–17, 2021, Virtual, Denmark

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8459-9/21/07...\$15.00

<https://doi.org/10.1145/3460319.3464818>

1 INTRODUCTION

Easy access to, and management of, data is essential for many mobile apps. Local databases, such as SQLite [18], store data directly on the mobile device. This improves the app's reliability and responsiveness, especially when the underlying mobile device does not have a reliable connection. These benefits have led to the widespread use of local databases. In fact, a recent study found that nearly 60% of mobile apps make use of local databases as part of their implementation [38].

Despite the benefits that local databases offer to developers, there are costs that come with database usage. Studies have found that local database services are among the top three resource-consuming services on mobile devices [32, 34]. Although all kinds of database interactions can be resource intensive, certain usage patterns can be especially problematic. According to a recent study [37], bad programming practices in using database operations, also known as SQL antipatterns, can have significant impact on the mobile app's resource consumption, including runtime and energy. In addition to performance issues, SQL antipatterns can undermine the security of mobile apps, for example by causing SQL injection or information leakage vulnerabilities [26, 66].

Although there is great value in detecting SQL antipatterns in mobile apps given the widespread use of local databases, designing a precise and efficient detector for this problem domain is nontrivial. The first challenge is the need to model SQL statements. Dynamically constructed string-based SQL statements are prevalent in modern mobile apps [38]. Such SQL statements are built through a sequence of string operations (like concatenation), and may vary across different code paths. Beyond SQL statements, there are other aspects that need to be modeled. As one example, an antipattern that concerns inefficient database writes requires reasoning about loops and transactions, which are challenging to model effectively [17].

The modeling needs that arise from the problem domain of local database usage, and the varying aspects that different antipatterns touch on (security, performance, reliability, and so on), motivate domain-specific abstractions. The goal of such abstractions is to encapsulate modeling complexities that stem from application/-database interactions, so that antipatterns can be specified easily and accurately.

We present SAND (SQL Antipattern Detection), an extensible analysis approach powered by a novel set of abstractions, which is able to check — with high accuracy and efficiency — for all known antipatterns. Thanks to the separation between specification of

SQL antipatterns and the underlying abstractions, SAND can be extended with additional antipatterns as these are discovered.

We have implemented a prototype version of SAND for the Android platform, where the abstractions take the form of Java types and methods. Our evaluation — consisting of all of the 11 SQL antipattern rules reported in a recent survey [37], and a suite of 1,000 Android apps — indicates that SAND is effective and efficient. The detection rules can be compactly expressed using 12–74 lines of fairly straightforward Java code, which validates the efficacy of the abstractions we chose and gives hope that future antipatterns can also be specified with relative ease. At the same time, the rules yield thousands of instances of SQL antipatterns in 38% of the subject apps with $\geq 99.4\%$ precision. SAND is able to identify more true positives than existing techniques given the same detection rules. Its performance is also encouraging. Running all of the 11 detectors requires an average of 41 seconds per app.

The structure of this paper is as follows. Background information is introduced in Section 2. The details of our abstractions are illustrated in Section 3 and Section 4. The detectors are explained in Section 5. The results of our evaluation are reported in Section 6. The threats to the validity of our evaluation are discussed in Section 7. Lastly, we discuss related work in Section 8 and conclude our paper in Section 9.

2 BACKGROUND

SQL is a domain-specific language used to access and manipulate databases. It is the dominant language to interact with databases [50], including in particular SQLite. SQL terminology that we use throughout the rest of the paper includes *projection*, which chooses a subset of columns (or expressions) according to criteria on their attributes, and *selection*, which ranges over rows.

In Table 1, we show a list of SQL antipatterns from the literature. The full definition, rationale, and related techniques for these SQL antipatterns can be found in a recent literature survey [37].

A local database reads from, and writes to, the device’s file system. In the Android ecosystem, SQLite has become the most common local database service. It is used in over 90% of all Android apps that make use of a local database service [38]. The Android runtime allows developers to manage a SQLite database using the class `SQLiteDatabase`. Two types of API signatures are provided in this class to interact with a database (read or modify its contents). The first type allows an entire SQL statement to be specified as a raw string that is passed as a parameter to the API, as shown at line 13 of Listing 1. The second type of signature predetermines the type of a database command, such as `SELECT`, and allows different parts of the command to be specified as arguments, as shown at line 4 of Listing 1. Internally, APIs that follow the second type combine the parameters through string concatenations and convert them into a complete SQL statement in the form of a raw string. We refer to the program locations that perform API invocations to issue SQL commands to the database as *database interaction points*.

There are two ways to select which columns to return in SQL. The first way is to explicitly specify the columns. For example, “`SELECT id, name, grade FROM students`”. The second way is to use the “`SELECT *`” syntax, which implicitly selects all the columns from the table. After issuing the query, cursors are used to store

the data retrieved from the database. Applications interact with the cursor object to iterate over the rows of the result (line 5 of Listing 1), and read the columns (line 7 of Listing 1). Retrieving a desired column from the cursor object is achieved by specifying the column index or column name. For example, at line 7 of Listing 1, the index zero is used to retrieve the first read column, *id*.

3 IDENTIFYING IMPORTANT DATABASE/APPLICATION RELATIONSHIPS

Our approach, SAND, provides abstractions for many different types of application/database interactions, which enable the concise and precise specification of SQL antipatterns that SAND can then detect. Given that new SQL antipatterns have been reported in the literature almost continuously from early 2005 to present times (e.g., [2, 5, 11, 27, 36, 39, 40, 49, 61, 62]), a key design goal is to define robust and widely applicable abstractions will enable SAND to detect known SQL antipatterns and provide for future extensibility. This goal is challenging since there are many types of, and variations on, SQL antipatterns. An appropriate set of abstractions must bridge across these variations to enable concise specifications. Therefore, a key contribution of this paper is in addressing the questions of what to abstract, and what abstractions to use, to detect SQL antipatterns effectively.

To address this challenge requires us to decide which application/database relationships should be represented as abstractions. These chosen abstractions ultimately determine the degree to which SAND is able to cover the full spectrum of SQL antipatterns, and so a rigorous process is needed. To this end, we analyzed the complete list of SQL antipatterns and their corresponding detectors, as described in a literature review [37]. These detectors, and their underlying algorithms, contain information about what to analyze and how to analyze the database access code. These are strong hints as to what should be included in the abstraction.

The process of identifying important application/database relationships can be divided into three steps. First, given a SQL antipattern, we identified the detection rules proposed by its existing detectors. These rules do not include the details of the detectors’ underlying analyses, but describe how those detectors judge whether a given database access code matches the antipattern. Second, given the detection rules, we extracted the application/database relationships that were analyzed by each rule. Finally, we categorized the extracted application/database relationships based on their type.

Following these three steps, we analyzed each of the SQL antipatterns and their detectors. The results of the first two steps are summarized in Table 1, where we show the detection rules and the corresponding application/database relationships. The results of the third step are summarized in Table 2. Each column represents one category of application/database relationships. Each row represents how an antipattern’s relationships (the items in the third column of Table 1) map to the general categories of application-database relationships. There are seven different kinds of application-database relationships. These are: (1) SQL statements; (2) data dependencies with respect to an SQL statement’s input and (3) output; (4) reachability relationships; (5) loops; (6) control dependencies; and (7) transactions. This set of important application/database relationships forms the basis for the design of the SAND abstractions.

Table 1: Antipatterns and their required abstracted information

Antipattern	Detection rules	Application/database relationships
Unbatched-Writes (UW)	Identify the database writes that are performed inside a loop but not inside an open transaction [11, 39].	(1) The type of the SQL statements (2) The control flow relationship between the program point that issues the SQL statement and the loops (3) The control flow relationship between the program point that issues the SQL statement and the open transactions
Not-Merging-Projection-Predicates (MPP)	Examine if two database reads can execute one after another, and the two issued queries are identical except for the projection predicates [2, 40].	(1) The type of the SQL statements (2) The reachability relationship between the two program points that issue the SQL statements (3) The concrete SQL statements
Not-Merging-Selection-Predicates (MSP)	Examine if two database reads can execute one after another, and the two issued queries are identical except for the selection predicates [2, 40].	(1) The type of the SQL statements (2) The reachability relationship between the two program points that issue the SQL statements (3) The concrete SQL statements
Loop-to-Join (LJ)	Check if a database read is performed to retrieve data, a loop then iterates using the result of the read data, and there is another database read issued in the same loop that executes after the previous database read and uses the previous read data [40].	(1) The type of the SQL statements (2) The data flow of the output of the SQL statements (3) The reachability relationship between the two program points that issue the SQL statements (4) The control flow relationship between the program point that issues the SQL statement and the loops
Vulnerable-Query (VQ)	Check if the concatenated input of the SQL statement is derived from a vulnerable input [36].	(1) The data flow of the input of the SQL statements
Not-Using-Parameterized-Query (PQ)	Analyze the parse structure of the SQL statement to identify substrings at the syntactical positions of data values, and determine if the values are generated dynamically [5].	(1) The data values of the SQL statements (2) The data flow of the input of the SQL statements
Not-Caching (NC)	Check if two database reads can execute one after another, and the two issued queries are equivalent [61].	(1) The type of the SQL statements (2) The reachability relationship between the program points that issue the SQL statements (3) The concrete SQL statements
Unnecessary-Column-Retrieval (UCR)	Find the database read, analyze the SQL statement to identify the columns retrieved by the read, and verify that there is no subsequent use of the retrieved columns [61].	(1) The type of the SQL statements (2) The projection predicates of the SQL statements (3) The data flow of the output of the SQL statements
Unnecessary-Row-Retrieval (URR)	Check if any data retrieved from a database read is used in a conditional construct (such as if), and this conditional construct guards the use of the retrieved data [15].	(1) The type of the SQL statements (2) The data flow of the output of the SQL statements (3) The control dependence relationship between the program points
Unbounded-Query (UQ)	Analyze the SQL statement to examine if it is bounded, such as using the LIMIT keyword [61], and check if there is any loop iterating over the output of this unbounded database read [49, 62].	(1) The concrete SQL statements (2) The data flow of the output of the SQL statements (3) The control flow relationship between the program points where the output is used and the loops
Readable-Password (RP)	Analyze the SQL statement to determine the part of the query that is likely to be sensitive, and then check this sensitive data has been encrypted [27].	(1) The concrete SQL statements (2) The data flow of the input of the SQL statements

Table 2: Categorized application/database relationships

Ptrn	SQL stmts	Input def	Output use	Reach	Loop	Ctrl dep	Transaction
UW	(1)				(2)		(3)
MPP	(1), (3)			(2)			
MSP	(1), (3)			(2)			
LJ	(1)		(2)	(3)	(4)		
VQ		(1)					
PQ	(1)	(2)					
NC	(1), (3)			(2)			
UCR	(1), (2)		(3)				
URR	(1)		(2)			(3)	
UQ	(1)		(2)		(3)		
RP	(1)	(2)					

4 DESIGN OF THE ABSTRACTIONS

Even with an appropriate set of application/database relationships identified, local database operations in mobile apps have complex semantics, which makes the design of the corresponding abstractions challenging as well. For example, applications frequently embed the construction of SQL statements in the application logic through the use of various string operations, complex code constructs, and long inter-procedural call chains [38]. To identify the application/-database relationships listed in Table 2, we defined abstractions and analyses to allow for the accurate and efficient identification of these abstractions in complex mobile code. In this section, we first explain *Silicas*, our abstraction for representing database interaction points and the possible SQL statements issued at those points, and then explain the set of abstractions and their realizing analyses that enable SAND to detect all of the application/database relationships identified in Table 2.

4.1 The Silica Abstraction

The silica abstraction serves as a fundamental unit of SAND. It is a tuple $\langle p, r \rangle$, consisting of a database interaction point p , and a string model r that represents the possible SQL statements issued at p . For the ease of explanation, given a silica s , we use $s.p$ and $s.r$ to denote the two elements in the tuple. We define S to be the set of all silicas in an Application Under Test (AUT), R to be the set of all models, D to be the set of all database interaction points, and P to be the set of all program statements. In order to identify and analyze the silicas, we define two functions.

4.1.1 Modeling SQL Statements. SAND models possible SQL statements issued in an AUT through the function $getSQLModel : p \in D \mapsto r \in R$. Given a database interaction point p , SAND normalizes the specific semantics of the database API invoked at p , and constructs a unified string model r that represents the possible SQL statements issued at p .

The normalization is important because of two reasons. First, different types of database APIs can be invoked at the database interaction points and each type has its specific semantics of constructing SQL statements. Second, the process of constructing SQL statements is very complex. In fact, two out of the top three most frequently used database APIs issue raw string-based SQL statements, half of which are constructed dynamically via complex string operations [38]. The SQL statements can be built across method boundaries and may vary over different paths of execution. However, most existing detectors either assume that the SQL statements are static/hard-coded [7, 40, 49, 55, 56], or that there is a one-to-one mapping between the database access API and the concrete SQL statement (i.e., Object-relational Mapping (ORM)) [10, 11, 13, 15, 61]. Some existing detectors develop their own mechanisms to model the string-based SQL statements [4, 5, 24], but suffer from slow performance, low precision, and the lack of ability to handle complex string operations. These drawbacks in string analysis ultimately affect the effectiveness and efficiency of the detectors.

To address the need of normalization, SAND bases the model r on an Intermediate Representation (IR) proposed by a string analysis technique, Violist [33]. The IR is structured as an expression tree with the leaf nodes defining either constants or placeholders for unknown variables. We represent expressions in the tree as $op\ a_1\ a_2$, where op represents the operator and a_i represents the operand. This IR captures data-flow dependencies in loops, context-sensitive call site information, and the string operations applied to the string variable along the various paths leading to the use of a string variable. In Listing 1, the IR of the parameter at line 13 is denoted as $+ \text{"INSERT INTO reg (id) VALUES ("} (+ X_7 \text{"})"$. The $+$ sign denotes string concatenation. The unique placeholder X_7 indicates that the variable that is external to the analysis scope is defined at line 7.

SAND extends Violist's analysis so that it can model the SQL statements issued by different types of database APIs. More specifically, SAND combines the semantics of various database APIs and constructs the model r accordingly. If an entire SQL statement is passed as a parameter to the API, the IR of the parameter is directly used as the value of r . If different parts of a SQL statement are passed as multiple parameters, SAND combines the semantics of the APIs with the IRs of these parameters, and converts them into a single IR. This single IR is used as the value of r . For instance, the

model r at line 4 of Listing 1 is $+ \text{"SELECT"} (+ \text{"id, name, grade"} + \text{"FROM"} (+ \text{"students"} (+ \text{"WHERE"} (+ \text{"name = " X}_3 \text{"}))))$.

SAND defines functionalities to represent r as a set of concrete finite strings, denoted as $\psi(r)$, that enables further analyses on the concrete SQL statements. At line 13 of Listing 1, $\psi(r) = \{\text{INSERT INTO reg (id) VALUES (X}_7)\}$. To generate $\psi(r)$, SAND interprets the expression tree represented by r . It performs a post-order traversal on the tree and applies the string operations in the internal nodes to their child nodes. The set of finite strings provided in $\psi(r)$ enables all kinds of analyses needed by the SQL antipattern detectors. As shown in Table 1, the detectors of seven SQL antipatterns require analyzing the concrete SQL statements, including their project predicates, selection predicates, data values, keywords, and so on. These detectors can leverage an existing SQL parser [58] to parse the statement and extract the parts that they are interested in. Subsequent analyses, such as string comparison, can be conducted according to the detectors' needs. Note that SAND does not require the concrete strings to be grammatically valid SQL. When the SQL parser fails to recognize the concrete strings, the detectors have the flexibility to perform the proper actions. For example, if a detector only needs to identify whether a SQL statement is a database write, it can simply analyze the string prefix and does not need to rely on a parser.

Although there could exist scenarios where parts of a SQL statement are derived from external inputs, whose values cannot be determined by SAND, they did not prevent the detectors built on top of SAND from precisely identifying the various SQL antipatterns. The first reason is that the unknown parts, marked by unique placeholders, are typically at the syntactical positions of table names, data values, etc and they do not affect the structure of the SQL statement. The parts of the SQL statement that were embedded in the code and were captured by SAND's static string analysis were sufficient for the detectors to carry out the necessary analyses on the SQL statements. In our experiment, we found that among 13,418 different database interaction points associated with the detected instances, 44% of them had at least one placeholder embedded in their corresponding r model. In our inspection of results, these placeholders did not affect the detection accuracy. The second reason is that even if the detector of certain antipatterns, such as Unnecessary-Column-Retrieval, needs to analyze the table names or column names, whose values can be derived from external sources, the placeholder mechanism allows the detectors to handle the situation accordingly. For example, a detector can treat an unknown column, marked by a placeholder, to be potentially any column. This allowed the detector to work precisely.

4.1.2 Identifying Silicas of Interest. We define a function, $find : regex \mapsto \{s \mid s \in S\}$, to identify silicas of interest via regular expressions (regexes). Given a regex, this function returns a set of silicas whose SQL statements match the given regex. Regexes are sufficient for the detectors to identify an initial set of silicas of interest. Although SQL is more expressive than a regular language, the detectors do not need to use regexes to express all possible SQL statements' forms. Instead, the detectors only need to identify certain patterns of SQL statements and all these patterns can be recognized by regexes. We were able to determine this by analyzing the detection rules in Table 1. For example, the detector of

Unbatched-Writes focuses on database writes, which can be recognized by the regex `(insert|update|delete).*`. The detectors of Unbounded-Query and Readable-Password target SQL statements that contain certain keywords. These SQL statements can be recognized by the regex `.*keyword.*`. The detector of Vulnerable-Query needs to identify all SQL statements, which can be recognized by the regex `.*`. The rest of the detectors focus on database reads and they can use the regex `select.*`.

To realize this function, SAND identified and analyzed each database interaction point p where $p \in D$. If $\exists q \in \psi(\text{getSQLModel}(p))$ (q matches regex), a silica tuple $\langle p, \text{getSQLModel}(p) \rangle$ will be added to the return set.

```

1 public void main()
2 {
3   String userInput = text.getText().toString();
4   Cursor cursor = database.query("students", new String[]{"id", "name", "grade"},
5     name = " + userInput, null, null, null, null);
6   while(cursor.moveToNext())
7   {
8     int id = cursor.getInt(0);
9     if(id < 10)
10    {
11      database.beginTransaction();
12      for(int i = 0; i < 10; i++)
13      {
14        database.execSQL("INSERT INTO reg (id) VALUES (" + id + ")");
15      }
16      database.endTransaction();
17    }
18  }

```

Listing 1: Example program

4.2 Other Abstractions

SAND provides an additional set of functions to identify the application/database relationships categorized in Section 4.2. These functions are expressive enough to specify all the required detection rules on the corresponding types of application/database relationships, and they help the detectors built on top of SAND effectively and efficiently analyze the database access code. For the ease of explanation, we illustrate the realization of the functions in the context of an intra-procedural analysis. In Section 4.4, we discuss how the analyses conducted by the functions work inter-procedurally and in the presence of activity lifecycle methods.

4.2.1 Modeling External Inputs Used to Construct the SQL Statement. The sources of data used to construct a SQL statement are computed through the function $\text{getDefSet} : s \in S \mapsto \{\langle f, \text{placeholder} \rangle \mid f \in P, \text{placeholder} \in \Sigma^*\}$. It takes as an argument a silica s , and returns a set of tuples, each of the form $\langle f, \text{placeholder} \rangle$. The label *placeholder* represents an external input to the AUT that is concatenated to a SQL statement, and the program point f indicates one of the possible definition points of this input. For example, given the silica s at line 13 of Listing 1, $\text{getDefSet}(s) = \{\langle \text{line 7}, X_7 \rangle\}$, indicating that an external input, labeled as X_7 , is defined at the program point at line 7. The label *placeholder* is used to associate the external input with its position in a SQL statement. For instance, $\psi(s.r) = \{\text{"INSERT INTO reg (id) VALUES (X}_7\text{)"}\}$ and X_7 is at the syntactical position of data values. This function enables a range of detection rules that reason about the input of a SQL statement. It is essential for the detection of three SQL antipatterns (Vulnerable-Query-1, Not-Using-Parameterized-Query-2, and Readable-Password-2).

Realizing this function requires SAND reasoning about the data flow of the variables that contribute to the construction of SQL statement. These inputs can follow different program paths across method boundaries and become parts of a SQL statements via complex string operations. Although there exist analyses that tried to identify syntactical positions of inputs in a SQL statement via symbolic execution [4, 5], the analyses could be inefficient and not scalable. To tackle the challenge, SAND utilizes its underlying full-power string analysis, which is a summary-based approach that is designed for high scalability. As described in Section 4.1, the string model r in the silica tuple $\langle p, r \rangle$ is an expression tree and it captures the data flow of variables. Every variable that is external to the analysis scope is represented by a unique placeholder in r . The position information about where the variables are defined is also incorporated in the placeholder. For instance, given the model r at line 13 of Listing 1, which is `"INSERT INTO reg (id) VALUES (" + X7 + ")"`, the placeholder is X_7 and the definition point is the program point at line 7. To implement the getDefSet function, SAND traverses the nodes in r and looks for the placeholders in the leaf nodes. For each identified placeholder h , SAND locates the program point f that corresponds to the definition point of this placeholder, and adds a tuple $\langle f, h \rangle$ to the return set of this function.

4.2.2 Modeling Usage of Outputs Returned by the SQL Statement.

The data flow of the output of a SQL statement is computed through the function $\text{getUseSet} : s \in S \mapsto \{\langle u, \text{column} \rangle \mid u \in P, \text{column} \in \Sigma^*\}$. This function takes as an argument a silica s , and returns a set of tuples, each of the form $\langle u, \text{column} \rangle$. The program point u indicates where the output data is used. The string label *column* associates u with the concrete column/expression of the output data. It represents one of the columns/expressions that are selected by the SQL query issued at $s.p$, and are used at the program point u . For instance, in Listing 1, given the silica at line 4, one of the return tuples is $\langle \text{line 8}, id \rangle$. The tuple means that the *id* column is used at line 8. This function allows the detectors to reason about the output of a SQL statement, including where the output is used, how the output is used, and which columns are used. It lays the foundation for the detection of three SQL antipatterns, Loop-to-Join-2, Unnecessary-Column-Retrieval-3, Unnecessary-Row-Retrieval-2, and Unbounded-Query-2.

There are two challenges regarding the realization of this function. First, the output data of a query is stored in a complex object, the cursor. Identifying the use of data not only requires tracking this cursor object, but also requires tracing any column data that is retrieved from it. Second, the cursor APIs provide flexible ways (i.e., via the column index or column name), to retrieve a desired column from the cursor object, making it challenging for SAND to associate the use of data with the columns statically. If the column index is used, the mapping from the index to the name can depend on the query and the corresponding table, because of the "SELECT *" syntax in SQL. If the column name is used, identifying its value is also challenging as the name is a string parameter whose value can be constructed inter-procedurally via string operations. Although there exist techniques that tried to identify the retrieved columns, they did not address the aforementioned challenges. The existing techniques either fail to do it statically [12], or do not handle the mappings from the column index to the column name [14, 61].

To address the first challenge, SAND develops a static taint analysis to track the information flow with respect to the cursor object [3]. SAND treats the cursor object returned by the database read as a taint source. During the taint propagation, if a column is retrieved, SAND performs a column identification analysis, which will be explained in the next paragraph, and annotates the tainted variable with the column name. For example, the variable *id* is tainted by the cursor object at line 7 of Listing 1. SAND annotates the variable *id* with the column name *id*. If the program point simply uses the cursor object but not any specific column data, such as line 5 in Listing 1, the annotation is *null*.

SAND's column identification analysis addresses the second challenge. Given a silica *s*, and a program point *u* where a column retrieval API is invoked (e.g., line 7 of Listing 1), the analysis returns a set of strings that represent the possible columns retrieved at *u*. Depending on whether a column name or a column index is used as the parameter of the column retrieval API, SAND carries out two different approaches. If a column name is used, SAND leverages a string analysis [33] to identify the possible values of the name, and uses them directly as the output of this column identification analysis. If a column index is used, after identifying its values, SAND maps the index to the name as explained below.

The mapping from the column index to the column name is determined by the query issued at the program point of the given silica *s*. As there are two ways to select columns in SQL, SAND handles the mapping accordingly. If the query explicitly selects columns, the mapping can be done by analyzing the query itself. More specifically, SAND parses the SQL statement, extracts the selected columns in order, and matches the index value to the corresponding column. For instance, given a query "SELECT *id*, *name*, *grade* FROM *students*" and an index zero, the index zero maps to the column *id*. If the query implicitly selects columns, the mapping requires analyzing not only the query, but also the table structure. To do that, SAND first parses the SQL statement and identifies the selected table. It then searches for the corresponding table creation statement by leveraging the *find* function. Next, SAND obtains the column order by parsing the table creation statement. Lastly, SAND matches the index value to the column. Using an illustrative example, the column index zero maps to the column *id* if the query is "SELECT * FROM *students*" and the table creation statement is "CREATE TABLE *students* (*id* INT, *name* TEXT, *grade* TEXT)". SAND currently assumes that table creation is done in the shipped app code. However, if an app comes with pre-created database tables, SAND does not analyze those tables yet and would make use a placeholder for the column name, as explained in the next paragraph. As a part of the future work, SAND can look into the pre-created tables stored in the assets of an app.

SAND annotates the tainted variable with a placeholder ω as the column name, when a column is selected but its name is unknown. Introducing the placeholder ω is necessary because of two reasons. First, the parameter of the column retrieval API can be derived from an external source. Second, the table creation statement is not in the app code, but identifying the column name requires analyzing the table. In those two cases, the column identification analysis is not able to determine the column name from code statically. The placeholder ω allows the detectors built on top of the abstraction to handle the unknown column based on their needs. For example,

they can safely assume that all the columns are selected because this unknown column can potentially be any column.

4.2.3 Modeling Reachability Relationships between SQL Statements. The reachability relationship is computed through the function $getReachableSet : p \in P \mapsto \{g \mid g \in P\}$. Given a program point *p*, the function returns a set of program points, $\{g \mid g \in P\}$, that are reachable from *p* in the Control Flow Graph (CFG). It satisfies the need of determining whether one database interaction point can execute after another, which is required by the detection of four SQL antipatterns (i.e., Not-Merging-Projection-Predicates-2, Not-Merging-Selection-Predicates-2, Loop-to-Join-3, and Not-Caching-2). To realize this function, SAND conducts a reachability analysis [1]. The analysis iteratively propagates the nodes in the CFG to their successors and converges when no more nodes can propagate to a new node. The nodes that the given program point can propagate to are returned by *getReachableSet*.

4.2.4 Modeling Control Flow Relationships between SQL Statements and Loops. The control flow relationship with respect to loops is computed through the function $getLoopSet : p \in P \mapsto \{l \mid l \in P\}$. It takes as an argument a program point *p*, and returns a set of program points, $\{l \mid l \in P\}$, which consists of the headers of the loops that enclose *p*. In Listing 1, the loops that contain the program point at line 13 have their headers at line 5 and line 11, respectively. This function addresses the need for expressing the detection rules of three SQL antipatterns (i.e., Unbatched-Writes-2, Loop-to-Join-4, and Unbounded-Query-3). To accurately identify nesting relationships between loops and the nodes that each loop contains, SAND employs a region-based analysis when realizing this function [1]. This analysis builds a Region Tree (RT) for each method by analyzing the dominators in a CFG. Each node in the RT represents a loop body and the nesting relationships between loops are modeled by the parent-child relationships in the RT.

4.2.5 Modeling Control Flow Relationships between SQL Statements and Conditional Statements. The control flow relationship with respect to conditional statements is computed through the function $getControlDependenceSet : p \in P \mapsto \{\langle c, i \rangle \mid c \in P, i \in \mathbb{Z}\}$. Given a program point *p*, this function returns a set of tuples, each of the form $\langle c, i \rangle$. The program point *c*, along with the integer *i*, represent which branch of an if/switch statement *p* is transitively control dependent on. For example, in Listing 1, line 13 is transitively control dependent on three conditional statements line 5, 8, and 11 when they evaluate to true. The return set is therefore $\{\langle \text{line 5}, 1 \rangle, \langle \text{line 8}, 1 \rangle, \langle \text{line 11}, 1 \rangle\}$, where the true branch is represented by the value 1. This function is needed by the detection of Unnecessary-Row-Retrieval. The detector can apply this function to the program points where the read data is retrieved and used. If there is a tuple that exists in all the return sets, it means that the given program points are guarded by the same branch of a conditional statement. To realize this function, SAND employs a control dependence analysis [16], which identifies the nodes in the CFG that the given program point is control dependent on.

4.2.6 Modeling Transactions. To identify the transactions that enclose the given database interaction point, we define a function, $getTransactionSet : p \in D \mapsto \{t \mid t \in P\}$. It takes an argument a database interaction point *p*, and returns a set of program points,

$\{t \mid t \in P\}$, where the transactions that enclose the given program point are initiated. In Listing 1, the database interaction point at line 13 is enclosed by the transaction initiated at line 10. The flexible semantics of transaction control introduce great challenges to realizing the *getTransactionSet* function. Thematically, analyzing transaction open and close operations is similar to detecting resource leakage, e.g., [21, 51]. However, a key difference is that for resource leakage, once the resource-releasing API is called, the corresponding resource is released. This is not true for nested transactions. If multiple transactions are open in a nested manner, the only way to close them is to issue the exact same number of transaction close operations. To accurately and efficiently analyze transactions, SAND utilizes a static analysis technique that is specialized at modeling nested transaction control [39]. Given a program point that issues a SQL statement, the analysis propagates this program point backwards in CFG, and uses a counter to track the invocations to the transaction open and close operations along the path leading to the given program point. Using the counter value, the analysis is able to judge whether a transaction is open or close at a certain program point. With this analysis, SAND identifies the transactions that remain open when the given program point executes. The program points where these transactions are open are added to the result set.

4.3 Inter-procedural Analysis

SAND’s inter-procedural analysis is precise and fast, which ensures the effectiveness and efficiency of detectors built on top of SAND. When performing inter-procedural analysis, SAND treats each activity lifecycle method as an entry point of the AUT. Within each activity lifecycle method, SAND uses the Cloned Call Graph (CCG) [45] of the AUT to perform the analyses inter-procedurally. In the CCG, every distinct calling context invokes a different instance of a method. This context-sensitive CCG improves precision and allows the analyses to work the same as if applied to an intra-procedural CFG. Although the CCG is generally quite large for even small programs, the design of the SAND allows it to prune the call graph of the AUT so that only the transitive callers of the silica containing methods and their transitive callees remain. The pruning process is done during the construction of silicas. After identifying the program point p in the silica tuple, SAND assumes that all abstractions can be used; it precomputes all related transitive callers and callees of the method that contains p . In our evaluation of 1,000 benchmarks, which are top-ranked apps in the Google Play App Store, we observed that unlike many other problem domains, local database accesses in Android applications often yield relatively sparse (albeit interprocedural) slices; this led to our analysis being able to prune 96% of the app’s code (in our 1,000 subject apps), which made the control-flow and data-flow analysis more efficient.

To further improve efficiency, SAND avoids redundant computation by caching analysis results. There are two kinds of caching mechanisms. First, the outputs of the functions are cached. The analysis results can be reused when multiple detectors that share a common set of functions and target silicas run on the same AUT. The second type of caching is to cache SAND’s intermediate analysis result on the CFG. Several analyses that the functions are based on, such as region analysis, have the same analysis results given the

same CFG. Such results can be cached and reused. For instance, if *getLoopSet* is called on two program points that are inside the same method, SAND can build the RT for this method once and reuse the RT. For another example, when computing *getUseSet*, after a method is visited, SAND caches the used columns and the program points that use them. The cached values are associated with the values of the tainted method parameters.

While it is possible to implement some of the abstractions with an IFDS approach [54], (e.g., we could compute some per-method data-flow facts based on all possible $\langle u, column \rangle$ pairs for *getUseSet*), we didn’t use IFDS in SAND for the following reasons. First, not all the abstractions can be encoded as IFDS problems. For example, transactions cannot be converted to a finite set of data-flow facts due to the flexible semantics of transaction operations in Android. Second, our approach is extensible and can accommodate different realization choices that would strengthen the implementation of data-flow related abstractions. For example, adding support for branched analysis and inter-callback data-flow relationships could mitigate certain false positives, as discussed later in Section 6.1.

4.4 Implementation

We implemented the abstractions as Java types and methods. The reason is that, first, Java is a language with which many people are familiar. Users do not need to learn a new language in order to use SAND. Second, Java is Turing-complete. The abstractions and functions can be used in combination with Java to express all kinds of analyses.

The current implementation of SAND is targeted to Android application binaries and their default local database management system, SQLite [18]. This implementation choice allowed us to evaluate SAND on a large number of marketplace applications, where SQLite is widely used [38]. Note that the design of SAND is generalizable. It can be extended to analyze other languages for which it is possible to generate CFGs and CCGs. It can also be extended to analyze other database APIs where SQL statements are strings, such as JDBC.

The analyses conducted on the AUT, such as building the CFG and Call Graph (CG), are based on the Soot analysis framework [30]. Soot helped to convert the applications’ binaries to its intermediate representation of bytecode, called *Jimple*. All the analyses conducted by SAND were based on the Jimple bytecode. Our implementation of SAND is available under the Apache Software License v2.0 from <https://github.com/USC-SQL/SAND>.

5 BUILDING SQL ANTIPATTERN DETECTORS USING SAND

To evaluate the efficacy of the abstractions powering SAND, and perform the most complete evaluation on our benchmark applications, we have implemented detectors for all the SQL antipatterns identified by a recent literature survey [37]. Real-world instances of these SQL antipatterns have been shown to significantly increase the resource consumption, or undermine the security of, mobile apps [37]. The detectors correspond to the rules listed in Table 1.

In the remainder of this section, we discuss the differentiation between SAND detectors and existing detection techniques, and

how it stems from the abstractions that we employ. Quantitative results and comparisons are provided in Section 6.1.

Unbatched-Writes (UW). For each silica that issues a database write, the script checks if the write is in a loop but not in a transaction, as shown in Equation (1). This SAND detector utilizes the detection logic proposed by the two techniques [11, 39]. However, both of the techniques cannot handle the dynamically constructed string-based SQL statements in mobile apps. Moreover, the technique by Chen *et al.* examines whether a database write is in a transaction by checking if an ORM annotation (*Batch*) exists [11]. This means that the technique cannot model the complex transaction operations in mobile apps as the SAND detector for Unbatched-Writes does.

$$\begin{aligned} T = \{s \mid s \in \text{find}(\text{"^(insert|update|delete).*"}) \\ \wedge \text{getLoopSet}(s.p) \neq \emptyset \\ \wedge \text{getTransactionSet}(s.p) = \emptyset\} \end{aligned} \quad (1)$$

Not-Merging-Projection-Predicates (MPP). The script first identifies the silicas that issue select queries and may execute one after another. For each pair of such silicas, the script then checks if the queries issued by the first and second silica in the pair are identical except for the projection predicates. It is done by string comparisons on the SQL statements returned by the ψ function. The set of instances of this antipattern is shown in Equation (2). The existing techniques for this antipattern either take the query log as an input and do not analyze the application code [2], or assume that the entire SQL statement is statically embedded at the database interaction point [40]. The SAND detector addresses these limitations.

$$\begin{aligned} T = \{ \langle s1, s2 \rangle \mid s1, s2 \in \text{find}(\text{"^select.*"}) \\ \wedge s2.p \in \text{getReachableSet}(s1.p) \\ \wedge \exists q1, q2 (q1 \in \psi(s1.r) \wedge q2 \in \psi(s2.r) \\ \wedge q1 \text{ and } q2 \text{ are identical except for the projection predicates}) \} \end{aligned} \quad (2)$$

Not-Merging-Selection-Predicates (MSP). The steps of detecting this antipattern is similar to the ones of detecting Not-Merging-Projection-Predicates. The difference is that in the detector, the script checks if the queries are identical except for the selection predicates. This detector addresses the same limitations of the existing techniques [2, 40] as discussed above.

Loop-to-Join (LJ). The script first identifies the pairs of silicas that issue select queries and may execute one after another. For each pair $\langle s1, s2 \rangle$, the script then checks if there is a loop that contains $s2.p$ and iterates using the result of $s1$, i.e., $\exists l (l \in \text{getLoopSet}(s2.p) \wedge \exists \langle u0, c0 \rangle (\langle u0, c0 \rangle \in \text{getUseSet}(s1) \wedge l = u0))$. Lastly, the script checks if the output of $s1$ is used at $s2$, i.e., $\exists \langle u1, c1 \rangle (\langle u1, c1 \rangle \in \text{getUseSet}(s1) \wedge u1 = s2.p)$. The set of instances of this antipattern is shown in Equation (3). The existing techniques for this antipattern either assume that the application uses ORM [13, 15] or the SQL statement is hardcoded [40].

$$\begin{aligned} T = \{ \langle s1, s2 \rangle \mid s1, s2 \in \text{find}(\text{"^select.*"}) \\ \wedge s2.p \in \text{getReachableSet}(s1.p) \\ \wedge \exists l (l \in \text{getLoopSet}(s2.p) \wedge \exists \langle p0, c0 \rangle (\langle u0, c0 \rangle \in \text{getUseSet}(s1) \\ \wedge l = u0)) \\ \wedge \exists \langle u1, c1 \rangle (\langle u1, c1 \rangle \in \text{getUseSet}(s1) \wedge u1 = s2.p) \} \end{aligned} \quad (3)$$

Vulnerable-Query (VQ). The script first locates all the silicas by using the regex `"^select.*"`, which represents any string. To identify if a tainted source is used to build the SQL statement, the script inspects each silica and checks if any input of the SQL statements represented the silica is returned by a tainted API. The set of instances of this antipattern is described in Equation (4). The script builder can decide the list of tainted APIs. For example, the list by Susi [53] can be used. It contains APIs whose returned values are considered to be untrusted. (This is also the list we use in our evaluation.)

$$\begin{aligned} T = \{s \mid s \in \text{find}(\text{"^select.*"}) \\ \wedge \exists \langle f, \text{placeholder} \rangle (\langle f, \text{placeholder} \rangle \in \text{getDefSet}(s) \\ \wedge f \text{ invokes a tainted API}) \} \end{aligned} \quad (4)$$

Not-Using-Parameterized-Query (PQ). The detection of this antipattern starts with finding all the silicas. For each silica, the script checks if any substring at the syntactical positions of data values is defined externally. The set of instances of this antipattern is described in Equation (5). Identifying data values is done by parsing the SQL statement q with a SQL parser [58].

$$\begin{aligned} T = \{s \mid s \in \text{find}(\text{"^select.*"}) \\ \wedge \exists \langle f, x \rangle (\langle f, x \rangle \in \text{getDefSet}(s) \wedge q \in \psi(s.r) \\ \wedge x \text{ is a data value of } q) \} \end{aligned} \quad (5)$$

Not-Caching (NC). The script first identifies the pairs of silicas that issue select queries and may execute one after another. It then verifies if the SQL statements issued by the two silicas in the pair are identical. This is done by string comparison on the SQL statements returned by the ψ function. The set of instances of this antipattern is shown in Equation (6). The existing technique for this antipattern only analyzes ORM applications and cannot analyze string-based SQL statements [61].

$$\begin{aligned} T = \{ \langle s1, s2 \rangle \mid s1, s2 \in \text{find}(\text{"^select.*"}) \\ \wedge s2.p \in \text{getReachableSet}(s1.p) \\ \wedge \exists q1, q2 (q1 \in \psi(s1.r) \wedge q2 \in \psi(s2.r) \wedge q1 = q2) \} \end{aligned} \quad (6)$$

Unnecessary-Column-Retrieval (UCR). For each silica s that issues a select query, let C denote the columns selected by the query q where $q \in \psi(s.r)$, and O denote the columns returned by $\text{getUseSet}(s)$. The script investigates if there exists any retrieved column that is not used, i.e., $\exists c \in C (c \notin O) \wedge \nexists o \in O (o = c)$. The purpose of checking if there exists any unknown column name, i.e., ω , is to avoid false-positives; because an unknown column means any possible column can be used. The set of instances of this antipattern is shown in Equation (7). Comparing to the SAND detector,

the two existing techniques cannot analyze string-based SQL statements [12, 61]. In addition, one of the techniques relies on dynamic analysis and cannot identify the used columns statically [12].

$$T = \{s \mid s \in \text{find}(\text{"^select.*"}) \wedge \exists c \in C(c \notin O) \wedge \nexists o \in O(o = \omega)\} \quad (7)$$

Unnecessary-Row-Retrieval (URR). For each silica s that issues a select query, the script checks if the following case exists. A selected column is used in an if statement, i.e., $\exists \langle u0, c0 \rangle (\langle u0, c0 \rangle \in \text{getUseSet}(s) \wedge u0 \text{ is an If statement} \wedge c0 \neq \text{null})$. The other program points that use the read data are control dependent on this if statement, i.e., $\forall \langle u, c \rangle (\langle u, c \rangle \in \text{getUseSet}(s) \wedge u \neq u0 \implies u0 \in \text{getControlDependenceSet}(p))$. The set of instances is described in Equation (8).

$$\begin{aligned} T = \{s \mid s \in \text{find}(\text{"^select.*"}) \\ \wedge \exists \langle u0, c0 \rangle (\langle u0, c0 \rangle \in \text{getUseSet}(s) \wedge u0 \text{ is an If statement} \\ \wedge c0 \neq \text{null} \wedge \forall \langle u, c \rangle (\langle u, c \rangle \in \text{getUseSet}(s) \\ \wedge u \neq u0 \implies u0 \in \text{getControlDependenceSet}(u)))\}. \end{aligned} \quad (8)$$

Unbounded-Query (UQ). The script starts with finding the select queries that may retrieve unbounded number of records. This is done by finding the silicas that match certain patterns, such as not using the keyword LIMIT. For each such silica, the script tests if there is a loop that iterates over the read data. The set of instances is shown in Equation (9). The existing techniques for this antipattern only analyze ORM applications and cannot analyze string-based SQL statements [61, 62].

$$\begin{aligned} T = \{s \mid s \in \text{find}(\text{"^select.*"}) \\ \wedge \exists q(q \in \psi(s.r) \wedge q \text{ is unbounded}) \\ \wedge \exists \langle u1, c1 \rangle, \langle u2, c2 \rangle (\langle u1, c1 \rangle \in \text{getUseSet}(s) \\ \wedge \langle u2, c2 \rangle \in \text{getUseSet}(s) \wedge u1 \in \text{getLoopSet}(u2))\} \end{aligned} \quad (9)$$

Readable-Password (RP). To detect this antipattern, the script first identifies the SQL statements that may include sensitive information. Then it searches for password comparisons by using the regex $(\text{"^.*password = .*"})$. Lastly, it checks if the value of the password has been encrypted, as shown in Equation (10). The existing study proposes a rule to identify this antipattern [27], but there is no existing detection technique for this antipattern.

$$\begin{aligned} T = \{s \mid s \in \text{find}(\text{"^.*password = .*"}) \\ \wedge \exists \langle f, x \rangle (\langle f, x \rangle \in \text{getDefSet}(s) \\ \wedge x \text{ is at the position after "password = " } \\ \wedge f \text{ invokes an encryption API})\} \end{aligned} \quad (10)$$

6 EVALUATION

To assess the effectiveness of SAND for SQL antipattern detection, we evaluated whether SAND can support a wide range of SQL antipattern detection tasks with concise scripts and good detection results. To assess the efficiency of SAND, we measured the time needed for the SAND detectors to run. We answered three research questions:

RQ1: What is the accuracy of the SAND detectors?

RQ2: How fast are the SAND detectors?

RQ3: What is the complexity of the detection scripts?

In order to measure the detection accuracy and speed, we ran the detectors on a set of subject apps. The subject pool contained 1,000 different marketplace Android applications. To ensure diversity in the subject pool, we selected apps that were: (1) downloaded frequently; (2) designed with different functional purposes (so that they covered different kinds of local database usage); (3) varied in terms of the amount of code. We identified potential subjects from the Google Play app store [19], which is the dominant app store where Android users download their apps. For each of the 34 categories defined by the store, we downloaded the top-ranked apps in the category and confirmed that the apps worked with Soot. After obtaining 1,000 subject apps through this process, we computed the amount of code that they had. The results showed that 16% of these apps had less than 10K bytecodes, 48% of them had bytecode counts between 10K and 100K, and 36% of them had more than 100K bytecodes. These numbers suggested that the apps were varied in terms of the size of app code.

6.1 RQ1: Detection Accuracy

6.1.1 Protocol. To answer RQ1, we focused on the true-positives and false-positives that the SAND detectors and existing techniques found in the subject apps. A detected instance is considered to be a true-positive if the instance conformed to the definition of the corresponding SQL antipattern. For instance, Unnecessary-Column-Retrieval defines an instance to be a database column being read without being used. A detected instance is considered to be a false-positive if all the read columns have been used. For each instance of a SQL antipattern detected, we decompiled the app and inspected the code of the methods in the call chain related to the detected instance so as to determine if the instance is a true-positive or false-positive. From the results of this analysis, we calculated the precision.

We were not able to obtain the number of false-negatives nor compute the recall. This is because different from calculating precision, calculating recall requires to analyze all of the bytecode to build the ground truth, instead of only examining the related code of the detected instances. It is extraordinarily difficult to accurately and manually analyze all of the bytecode for the large set of subject apps to build the ground truth.

As a baseline, we compared the accuracy of the SAND detectors against the original detector techniques defined in the literature. However, many of the original techniques did not make their tools available or targeted different programming languages (Ruby, PHP, Java, etc) and platforms. To address this issue, we modified SAND's underlying analyses to match the assumptions that the original techniques made about the characteristics of the database access code. For example, some SQL antipatterns detectors assumed that the SQL statements were defined as hard-coded strings or that there was a one-to-one mapping from the API to the SQL statement. We configured SAND as needed and ran these modified detectors on the same set of subject apps and compared the detection results with the full-featured SAND detectors. In this way, we were able to evaluate the value of the SAND design of the abstractions on the detection accuracy.

Table 3: Analysis results

Antipat- tern	# TPs/FPs (SAND)	# DB points	# Apps	# TPs/FPs (Modified)
UW	990/0	990/0	151/0	943/0
MPP	262/0	156/0	15/0	54/0
MSP	8,696/48	508/32	21/1	21/0
LJ	111/0	173/0	20/0	96/0
VQ	2,398/0	2,398/0	78/0	N/A
PQ	1,748/0	1,748/0	189/0	N/A
NC	5,961/4	1,144/8	80/4	4,478/4
UCR	2,939/18	2,939/18	270/6	1,871/12
URR	108/0	108/0	35/0	N/A
UQ	1,517/0	1,517/0	319/0	858/0
RP	53/0	53/0	8/0	N/A

6.1.2 Result. The detection results are summarized in Table 3. For each SQL antipattern, this table provides the number of detected true-positives (TP) and false-positives (FP) for both the full-featured SAND detectors and the modified versions. If the original detector techniques for a SQL antipattern did not make assumptions about how SQL statements are constructed, we marked "N/A" in the column, as there would be no difference in detection accuracy. The table also lists the number of distinct database interaction points associated with these TPs and FPs and the number of apps that contain these TPs and FPs.

6.1.3 Discussion. Overall, the SAND's detection precision ranged from 99.4% to 100%. In total, 383 out of the 1,000 subjects contained at least one SQL antipattern. These results demonstrate SAND's effectiveness in accurately detecting SQL antipatterns.

In addition to showing the accuracy of the SAND based detectors, the results also show the usefulness of the design of the analyses that underpin the SAND abstractions. This can be seen in the significantly better detection results the full-featured detectors had versus the implementations representing the original detector analyses. For example, the full-featured SAND detector identified 262 instances of Not-Merging-Projection-Predicates. The original defined version assumed all queries would be hard-coded strings [40], and only identified 54 instances (almost 80% less). The reason for the good performance of the full-featured SAND detectors is that the abstractions were well designed and can handle the complex database access code in mobile apps. The statistics we collected based on the subject apps showed that 94% of the database interaction points associated with the detected instances were inter-procedural and had an average of 4 methods involved in the call chain. In addition, only 13% of detected database interaction points have the entire SQL statement hard coded as a raw string. In most cases, the full-featured detectors performed significantly better. The only exception to this was Unbatched-Writes. This was due to the fact that the rule only needed to know whether a database write was performed when it reasoned about SQL statements [39]. In Android SQLite, usually it is sufficient to analyze the signature of the API to determine whether the API issues a database write or not. Therefore, even if the original version of the detector could not analyze raw-string SQL statements, it did not identify fewer instances.

To better understand the limitations of SAND, we investigated cases where the precision was less than 100%. We found that these false-positives were due to two limitations of SAND's underlying techniques. The first limitation is that SAND does not model inter-callback control-flow and inter-callback data-flow relationships. SAND treats each event handler callback and lifecycle callback as an entry point of the AUT and assumes that the callbacks are independent of each other. As a result of this, the detectors could not trace data that was used in another lifecycle method, which led to false-positives while detecting Unnecessary-Column-Retrieval. Analyzing relationships between callbacks is still an open problem in program analysis. Existing research efforts focus on determining the execution orders between GUI-related callbacks [63, 64], constructing callback summaries based on Android API methods [8, 52], or assuming that callbacks can be executed in any arbitrary order [3]. The results of these analyses could be integrated into SAND so as to further improve the detection results. Second, the string analysis that SAND used was safe but imprecise in handling conditional string values. For example, if there was a branch statement that could provide a value of a string, the string analysis considered all the possible branches as feasible. This imprecision ultimately led to several false-positives in detecting Not-Caching and Not-Merging-Selection-Predicates. If the string analysis were to be extended in a way that could eliminate infeasible paths, such false-positives would be eliminated.

6.2 RQ2: Analysis Time of the SAND Detectors

To evaluate the speed of the SAND detectors in finding SQL antipatterns, we calculated the execution time of the eleven detectors running on the subject apps. This execution time included the entire detection process from converting the apps' bytecode to conducting various analyses. To validate the contribution of the caching mechanism in improving the execution time of SAND detectors, we ran the detectors on the same set of apps while disabling the caching mechanism. The experiment result showed that the average and median execution time of running all the SAND detectors on one app were 41 seconds and 21 seconds, respectively. For 92% of the apps, all the detection analyses were finished within 60 seconds. When breaking down the total execution time, we found that 42% of the total time (18 seconds on average) was consumed by Soot converting bytecodes, and 14% of the total time (6 seconds on average) was consumed by Violist conducting the string analysis. If the caching mechanism was disabled, the average execution time of running all detectors went up to 289 seconds, seven times longer than the execution time of running the detectors with caching enabled. The median execution time was similar (22 seconds). Taken together these numbers suggest that the SAND detectors were efficient in analyzing modern marketplace mobile apps. The caching mechanism had played an important role in ensuring the efficiency of the SAND detectors. The time consumed by Violist demonstrated that this full power string analysis used by SAND helped the detectors identify more true-positives of various SQL antipatterns (as shown in Section 6.1); but it did not come at the cost of significant runtime overhead.

Table 4: Statistics of detection scripts

Antipattern	# Predicates	LOC	Complexity
UW	3	12	4
MPP	3	25	11
MSP	3	25	11
LJ	7	45	13
VQ	3	18	5
PQ	4	25	7
NC	5	21	8
UCR	6	29	8
URR	7	52	14
UQ	4	74	19
RP	4	29	7

6.3 RQ3: Code Complexity of SAND Detectors

To answer RQ3, for each of the SAND detector, we counted the number of predicates needed for specifying the detection rules, the number of lines of Java code (LOC) in the script, and the cyclomatic complexity of the code in the script [42].

The results are summarized in Table 4. The number of predicates ranges from 3 to 7. The LOC of the SAND scripts range from 12 to 74. The mean of the LOC is 32 and the median is 25. Among the 11 scripts, 10/11 of them have less than or near 50 LOC. The detector of Unbounded-Query requires 74 LOC because the detection rules proposed by the technique check several properties on the query itself in order to determine that the query is unbounded [61]. Processing the query string needed around 40 lines of code. In addition to being compact, the code also has low complexity. Studies on cyclomatic complexity suggest different values (10, 15, and 20) as the thresholds between acceptable and complex [42, 60, 65]. As shown in Table 4, 6/11 of the scripts have complexity values within 10; 10/11 of the scripts have complexity values within 15; and all of the scripts have complexity values within 20. These numbers demonstrate that the complexity values of most of the scripts are within or close to the threshold. Although there is one script that exceeds the threshold of 10 and 15, we found that the high complexity is due to the processing code of the query string when detecting Unbounded-Query. If we do not take into account this part of the code when computing the cyclomatic complexity, the complexity value of the Unbounded-Query detector would be 9. Based on these results, the detection rules can be compactly expressed using SAND’s abstractions.

7 THREATS TO VALIDITY

The effectiveness of SAND in expressing SQL antipattern detection tasks is based on the assumption that the list of SQL antipatterns we used is representative. In order to avoid biases, and maximize the completeness of the list, we used all SQL antipatterns presented in a recent literature survey [37], which followed best practices in literature review [29] to explore the current knowledge about SQL antipatterns.

SAND is extensible, so missed or future SQL antipatterns can be added in the future as long as our vocabulary of abstractions is adequate. Our ability to achieve high precision on all these known

rules is encouraging. It serves as evidence that the abstractions are effective and expressive, and also indicates that rules themselves are well defined.

8 RELATED WORK

The most similar work to ours is the work by Dasgupta *et al.* [14]. Their approach provides a set of services that analyze the database applications. Our approach is different from theirs in the following aspects. First, their approach analyzes a subset of application/database relationships that SAND analyzes. The reachability relationships, loops, transactions, etc, which are essential to the detection of many SQL antipatterns, are not provided in their approach. This may due to the lack of the process of distilling application/database relationships from a comprehensive list of SQL antipatterns and detectors in their approach. Second, their approach only handles string concatenation and simple control flow, while ours supports all of the string operations in the Java API and various complex control flow.

There are many approaches that first extract SQL statements from a program, then carry out different kinds of detection or repair on the extracted SQL statements. For example, Nagy *et al.* proposed searching mechanisms on the SQL statements [48], and proposed analyses to identify code smells in the queries [46, 47]. Guo *et al.* identified opportunities to repair SQL faults [22, 23]. Vásquez *et al.* and Li *et al.* proposed analyses to identify violations of schema constraints [31, 35]. SAND is different from these approaches in two main aspects. First, their approaches cannot model complex string operations and control flow. Second they mainly focus on SQL statements while SAND targets a variety of application/database relationships in addition to the SQL statements.

A variety of program analysis frameworks have been proposed to help people conduct sophisticated program analyses. The aims of these frameworks include but are not limit to symbolic execution [6], refactoring [59], instrumentation [25], detecting privacy leaks [28], and checking design rules [41, 44]. However, these frameworks do not target application-DMBS relationships.

Researchers and practitioners have focused on helping developers detect SQL antipatterns in their applications. One group of detectors relies on dynamic profiling and logging, e.g., [2, 9, 12, 43, 57]. These techniques analyze the logged SQL statements and other useful information to identify the occurrence of certain SQL antipatterns. SAND is a static analysis approach and thus it is different from this group of detectors. Another group of detectors leverages static analyses, e.g., [10, 11, 13, 15, 20, 40, 49, 61]. The difference between SAND and these static detectors is that SAND is an extensible approach and is not tailored towards specific antipatterns.

9 CONCLUSIONS

Ensuring proper usage of local database operations is critical for mobile apps. In the paper, we propose a static analysis approach, SAND, to detect SQL antipatterns in mobile apps. SAND features novel abstractions for common forms of application/database interactions, which enables concise and precise specification of the SQL antipatterns that SAND checks for. We evaluated eleven detectors built on top of SAND on a set of 1,000 marketplace Android apps.

The detectors finished the analyses in 41 seconds with a detection precision of at least 99%.

ACKNOWLEDGMENTS

This work was supported, in part, by the Office of Naval Research under grant number N00014-17-1-2896.

REFERENCES

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. 2006. *Compilers: Principles, Techniques, and Tools (2Nd Edition)*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA.
- [2] N. Arzamasova, M. Schäler, and K. Böhm. 2018. Cleaning Antipatterns in an SQL Query Log. *IEEE Transactions on Knowledge and Data Engineering* 30, 3 (March 2018), 421–434. <https://doi.org/10.1109/TKDE.2017.2772252>
- [3] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. FlowDroid: Precise Context, Flow, Field, Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Edinburgh, United Kingdom) (*PLDI '14*). ACM, New York, NY, USA, 259–269. <https://doi.org/10.1145/2594291.2594299>
- [4] Prithvi Bisht, A. Prasad Sistla, and VN Venkatakrishnan. 2010. Automatically preparing safe SQL queries. In *International Conference on Financial Cryptography and Data Security*. Springer, 272–288.
- [5] Prithvi Bisht, A. Prasad Sistla, and V. N. Venkatakrishnan. 2010. TAPS: Automatically Preparing Safe SQL Queries. In *Proceedings of the 17th ACM Conference on Computer and Communications Security* (Chicago, Illinois, USA) (*CCS '10*). ACM, New York, NY, USA, 645–647. <https://doi.org/10.1145/1866307.1866384>
- [6] Bernd Burgstaller, Bernhard Scholz, and Johann Blieberger. 2012. A symbolic analysis framework for static analysis of imperative programming languages. *Journal of Systems and Software* 85, 6 (2012), 1418–1439.
- [7] Wei Cao and Dennis Shasha. 2013. AppSleuth: a tool for database tuning at the application level. In *Proceedings of the 16th International Conference on Extending Database Technology*. ACM, 589–600.
- [8] Yinzhi Cao, Yanick Fratantonio, Antonio Bianchi, Manuel Egele, Christopher Kruegel, Giovanni Vigna, and Yan Chen. 2015. EdgeMiner: Automatically Detecting Implicit Control Flow Transitions through the Android Framework. In *22nd Annual Network and Distributed System Security Symposium, NDSS 2015, San Diego, California, USA, February 8-11, 2015*. The Internet Society. <https://www.ndss-symposium.org/ndss2015/edgeminer-automatically-detecting-implicit-control-flow-transitions-through-android-framework>
- [9] Surajit Chaudhuri, Vivek Narasayya, and Manoj Syamala. 2007. Bridging the Application and DBMS Profiling Divide for Database Application Developers. In *Vldb. Very Large Data Bases Endowment Inc.*
- [10] Tse-Hsun Chen, Weiyl Shang, Ahmed E Hassan, Mohamed Nasser, and Parminder Flora. 2016. CacheOptimizer: Helping developers configure caching frameworks for Hibernate-based database-centric web applications. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. ACM, 666–677.
- [11] Tse-Hsun Chen, Weiyl Shang, Zhen Ming Jiang, Ahmed E. Hassan, Mohamed Nasser, and Parminder Flora. 2014. Detecting Performance Anti-patterns for Applications Developed Using Object-relational Mapping. In *Proceedings of the 36th International Conference on Software Engineering* (Hyderabad, India) (*ICSE 2014*). ACM, New York, NY, USA, 1001–1012. <https://doi.org/10.1145/2568225.2568259>
- [12] T. H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. Nasser, and P. Flora. 2016. Finding and Evaluating the Performance Impact of Redundant Data Access for Applications that are Developed Using Object-Relational Mapping Frameworks. *IEEE Transactions on Software Engineering* 42, 12 (Dec 2016), 1148–1161. <https://doi.org/10.1109/TSE.2016.2553039>
- [13] Alvin Cheung, Armando Solar-Lezama, and Samuel Madden. 2013. Optimizing Database-backed Applications with Query Synthesis. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Seattle, Washington, USA) (*PLDI '13*). ACM, New York, NY, USA, 3–14. <https://doi.org/10.1145/2491956.2462180>
- [14] Arjun Dasgupta, Vivek Narasayya, and Manoj Syamala. 2009. A Static Analysis Framework for Database Applications. In *Proceedings of the 2009 IEEE International Conference on Data Engineering (ICDE '09)*. IEEE Computer Society, Washington, DC, USA, 1403–1414. <https://doi.org/10.1109/ICDE.2009.98>
- [15] K. Venkatesh Emani, Tejas Deshpande, Karthik Ramachandra, and S. Sudarshan. 2017. DBridge: Translating Imperative Code to SQL. In *Proceedings of the 2017 ACM International Conference on Management of Data* (Chicago, Illinois, USA) (*SIGMOD '17*). ACM, New York, NY, USA, 1663–1666. <https://doi.org/10.1145/3035918.3058747>
- [16] Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. 1987. The Program Dependence Graph and Its Use in Optimization. *ACM Trans. Program. Lang. Syst.* 9, 3 (July 1987), 319–349. <https://doi.org/10.1145/24039.24041>
- [17] Github. 2020. RAT-TRAP: A Tool of Automated Optimization of Resource Inefficient Database Writes for Mobile Applications. <https://github.com/USC-SQL/RAT-TRAP>.
- [18] Google. 2019. Android SQLite Documentation. <https://developer.android.com/reference/android/database/sqlite/SQLiteDatabase.html>.
- [19] Google. 2019. Gooogle Play App Store. <https://play.google.com/store/apps>.
- [20] Carl Gould, Zhendong Su, and Premkumar Devanbu. 2004. JDBC Checker: A Static Analysis Tool for SQL/JDBC Applications. In *Proceedings of the 26th International Conference on Software Engineering (ICSE '04)*. IEEE Computer Society, Washington, DC, USA, 697–698. <http://dl.acm.org/citation.cfm?id=998675.999476>
- [21] Chaorong Guo, Jian Zhang, Jun Yan, Zhiqiang Zhang, and Yanli Zhang. 2013. Characterizing and detecting resource leaks in Android applications. In *Automated Software Engineering (ASE), 2013 IEEE/ACM 28th International Conference on*. 389–398. <https://doi.org/10.1109/ASE.2013.6693097>
- [22] Y. Guo. 2017. Localizing and Fixing Faults in SQL Predicates. In *2017 IEEE International Conference on Software Testing, Verification and Validation (ICST)*. 555–556. <https://doi.org/10.1109/ICST.2017.72>
- [23] Y. Guo, N. Li, J. Offutt, and A. Motro. 2018. Automatically Repairing SQL Faults. In *2018 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. 500–511. <https://doi.org/10.1109/QRS.2018.00063>
- [24] William G. J. Halfond and Alessandro Orso. 2006. Preventing SQL Injection Attacks Using AMNESIA. In *Proceedings of the 28th International Conference on Software Engineering (Shanghai, China) (ICSE '06)*. ACM, New York, NY, USA, 795–798. <https://doi.org/10.1145/1134285.1134416>
- [25] Shuai Hao, Ding Li, William G. J. Halfond, and Ramesh Govindan. 2013. SIF: A Selective Instrumentation Framework for Mobile Applications. In *Proceedings of the 11th International Conference on Mobile Systems, Applications and Services (MobiSys)*.
- [26] Roei Hay, Omer Tripp, and Marco Pistoia. 2015. Dynamic Detection of Inter-application Communication Vulnerabilities in Android. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis* (Baltimore, MD, USA) (*ISSTA 2015*). ACM, 118–128.
- [27] Bill Karwin. 2010. *SQL Antipatterns: Avoiding the Pitfalls of Database Programming* (1st ed.). Pragmatic Bookshelf.
- [28] Jinyung Kim, Yongho Yoon, and Kwangkeun Yi. 2012. SCANDAL : Static Analyzer for Detecting Privacy Leaks in Android Applications.
- [29] Barbara Kitchenham. 2004. Procedures for Performing Systematic Reviews. 33 (08 2004).
- [30] Patrick Lam, Eric Bodden, Ondrej Lhoták, and Laurie Hendren. 2011. The Soot framework for Java program analysis: a retrospective. In *Cetus Users and Compiler Infrastructure Workshop (CETUS 2011)*.
- [31] B. Li, D. Poshvyanyk, and M. Grechanik. 2017. Automatically Detecting Integrity Violations in Database-Centric Applications. In *2017 IEEE/ACM 25th International Conference on Program Comprehension (ICPC)*. 251–262. <https://doi.org/10.1109/ICPC.2017.37>
- [32] Ding Li, Shuai Hao, Jiaping Gui, and William G.J. Halfond. 2014. An Empirical Study of the Energy Consumption of Android Applications. In *Proceedings of the International Conference on Software Maintenance and Evolution (ICSME)*.
- [33] Ding Li, Yingjun Lyu, Mian Wan, and William G. J. Halfond. 2015. String Analysis for Java and Android Applications. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (Bergamo, Italy) (ESEC/FSE 2015)*. ACM, New York, NY, USA, 661–672. <https://doi.org/10.1145/2786805.2786879>
- [34] Mario Linares-Vásquez, Gabriele Bavota, Carlos Bernal-Cárdenas, Rocco Oliveto, Massimiliano Di Penta, and Denys Poshyvanyk. 2014. Mining energy-greedy API usage patterns in Android apps: an empirical study. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*.
- [35] Mario Linares-Vásquez, Boyang Li, Christopher Vendome, and Denys Poshyvanyk. 2016. Documenting Database Usages and Schema Constraints in Database-Centric Applications. In *Proceedings of the 25th International Symposium on Software Testing and Analysis* (Saarbrücken, Germany) (*ISSTA 2016*). Association for Computing Machinery, New York, NY, USA, 270–281. <https://doi.org/10.1145/2931037.2931072>
- [36] V. Benjamin Livshits and Monica S. Lam. 2005. Finding Security Vulnerabilities in Java Applications with Static Analysis. In *Proceedings of the 14th Conference on USENIX Security Symposium - Volume 14* (Baltimore, MD) (*SSYM'05*). USENIX Association, Berkeley, CA, USA, 18–18. <http://dl.acm.org/citation.cfm?id=1251398.1251416>
- [37] Y. Lyu, A. Alotaibi, and W. G. J. Halfond. 2019. Quantifying the Performance Impact of SQL Antipatterns on Mobile Applications. In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 53–64.
- [38] Yingjun Lyu, Jiaping Gui, Mian Wan, and William G. J. Halfond. 2017. An Empirical Study of Local Database Usage in Android Applications. In *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 444–455. <https://doi.org/10.1109/ICSME.2017.75>

- [39] Yingjun Lyu, Ding Li, and William G. J. Halfond. 2018. Remove RATs from Your Code: Automated Optimization of Resource Inefficient Database Writes for Mobile Applications. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Amsterdam, Netherlands) (ISSTA 2018). ACM, New York, NY, USA, 310–321. <https://doi.org/10.1145/3213846.3213865>
- [40] A. Manjhi, C. Garrod, B. M. Maggs, T. C. Mowry, and A. Tomic. 2009. Holistic Query Transformations for Dynamic Web Applications. In *2009 IEEE 25th International Conference on Data Engineering*. 1175–1178. <https://doi.org/10.1109/ICDE.2009.194>
- [41] Michael Martin, Benjamin Livshits, and Monica S. Lam. 2005. Finding Application Errors and Security Flaws Using PQL: A Program Query Language. In *Proceedings of the 20th Annual ACM SIGPLAN Conference on Object-oriented Programming, Systems, Languages, and Applications* (San Diego, CA, USA) (OOPSLA '05). ACM, New York, NY, USA, 365–383. <https://doi.org/10.1145/1094811.1094840>
- [42] Thomas J McCabe. 1976. A complexity measure. *IEEE Transactions on software Engineering* 4 (1976), 308–320.
- [43] Phil McMinn, Chris J. Wright, and Gregory M. Kapfhammer. 2015. The Effectiveness of Test Coverage Criteria for Relational Database Schema Integrity Constraints. *ACM Trans. Softw. Eng. Methodol.* 25, 1, Article 8 (Dec. 2015), 49 pages. <https://doi.org/10.1145/2818639>
- [44] Clint Morgan, Kris De Volder, and Eric Wohlstadter. 2007. A Static Aspect Language for Checking Design Rules. In *Proceedings of the 6th International Conference on Aspect-oriented Software Development* (Vancouver, British Columbia, Canada) (AOSD '07). ACM, New York, NY, USA, 63–72. <https://doi.org/10.1145/1218563.1218571>
- [45] Steven S. Muchnick. 1997. *Advanced Compiler Design Implementation*. Morgan Kaufmann.
- [46] C. Nagy and A. Cleve. 2017. A Static Code Smell Detector for SQL Queries Embedded in Java Code. In *2017 IEEE 17th International Working Conference on Source Code Analysis and Manipulation (SCAM)*. 147–152. <https://doi.org/10.1109/SCAM.2017.19>
- [47] C. Nagy and A. Cleve. 2018. SQLInspect: A Static Analyzer to Inspect Database Usage in Java Applications. In *2018 IEEE/ACM 40th International Conference on Software Engineering: Companion (ICSE-Companion)*. 93–96.
- [48] C. Nagy, L. Meurice, and A. Cleve. 2015. Where was this SQL query executed? a static concept location approach. In *2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. 580–584. <https://doi.org/10.1109/SANER.2015.7081881>
- [49] Oswaldo Olivo, Isil Dillig, and Calvin Lin. 2015. Detecting and Exploiting Second Order Denial-of-Service Vulnerabilities in Web Applications. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security* (Denver, Colorado, USA) (CCS '15). ACM, New York, NY, USA, 616–628. <https://doi.org/10.1145/2810103.2813680>
- [50] Stack Overflow. 2018. 2018 Survey of StackOverflow. <https://insights.stackoverflow.com/survey/2018>.
- [51] Abhinav Pathak, Abhilash Jindal, Y Charlie Hu, and Samuel P Midkiff. 2012. What is keeping my phone awake?: characterizing and detecting no-sleep energy bugs in smartphone apps. In *MobiSys*.
- [52] Danilo Dominguez Perez and Wei Le. 2017. Generating Predicate Callback Summaries for the Android Framework. In *Proceedings of the 4th International Conference on Mobile Software Engineering and Systems* (Buenos Aires, Argentina) (MOBILESoft '17). IEEE Press, 68–78. <https://doi.org/10.1109/MOBILESoft.2017.28>
- [53] Siegfried Rasthofer, Steven Arzt, and Eric Bodden. 2014. A Machine-learning Approach for Classifying and Categorizing Android Sources and Sinks. In *Proceedings of the 21th Annual Network and Distributed System Security Symposium (NDSS'14)*. San Diego, CA.
- [54] Thomas Reps, Susan Horwitz, and Mooly Sagiv. 1995. Precise Interprocedural Dataflow Analysis via Graph Reachability. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Francisco, California, USA) (POPL '95). Association for Computing Machinery, New York, NY, USA, 49–61. <https://doi.org/10.1145/199448.199462>
- [55] Ziv Scully and Adam Chlipala. 2017. A program optimization for automatic database result caching. *ACM SIGPLAN Notices* 52, 1 (2017), 271–284.
- [56] Tushar Sharma, Marios Fragkoulis, Stamatis Rizou, Magiel Bruntink, and Diodemis Spinellis. 2018. Smelly Relations: Measuring and Understanding Database Schema Quality. (2018).
- [57] Juan M. Tamayo, Alex Aiken, Nathan Bronson, and Mooly Sagiv. 2012. Understanding the Behavior of Database Operations Under Program Control. In *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications* (Tucson, Arizona, USA) (OOPSLA '12). ACM, New York, NY, USA, 983–996. <https://doi.org/10.1145/2384616.2384688>
- [58] Tobias. 2017. JSQLParser. <https://github.com/JSQLParser/JSqlParser>.
- [59] Mathieu Verbaere, Ran Ettinger, and Oege de Moor. 2006. JunGL: A Scripting Language for Refactoring. In *Proceedings of the 28th International Conference on Software Engineering* (Shanghai, China) (ICSE '06). ACM, New York, NY, USA, 172–181. <https://doi.org/10.1145/1134285.1134311>
- [60] Arthur Henry Watson, Dolores R Wallace, and Thomas J McCabe. 1996. *Structured testing: A testing methodology using the cyclomatic complexity metric*. Vol. 500. US Department of Commerce, Technology Administration, National Institute of ...
- [61] Cong Yan, Alvin Cheung, Junwen Yang, and Shan Lu. 2017. Understanding Database Performance Inefficiencies in Real-world Web Applications. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management* (Singapore, Singapore) (CIKM '17). ACM, New York, NY, USA, 1299–1308. <https://doi.org/10.1145/3132847.3132954>
- [62] Junwen Yang, Cong Yan, Chengcheng Wan, Shan Lu, and Alvin Cheung. 2019. View-centric Performance Optimization for Database-backed Web Applications. In *Proceedings of the 41st International Conference on Software Engineering* (Montreal, Quebec, Canada) (ICSE '19). IEEE Press, Piscataway, NJ, USA, 994–1004. <https://doi.org/10.1109/ICSE.2019.00104>
- [63] Shengqian Yang, Haowei Wu, Hailong Zhang, Yan Wang, Chandrasekar Swaminathan, Dacong Yan, and Atanas Rountev. 2018. Static Window Transition Graphs for Android. *Automated Software Engg.* 25, 4 (Dec. 2018), 833–873.
- [64] Shengqian Yang, Dacong Yan, Haowei Wu, Yan Wang, and Atanas Rountev. 2015. Static Control-Flow Analysis of User-Driven Callbacks in Android Applications. In *Proceedings of the 37th International Conference on Software Engineering - Volume 1* (Florence, Italy) (ICSE '15). IEEE Press, 89–99.
- [65] Sheng Yu and Shijie Zhou. 2010. A survey on metric of software complexity. In *2010 2nd IEEE International Conference on Information Management and Engineering*. IEEE, 352–356.
- [66] Mu Zhang and Heng Yin. 2014. AppSealer: Automatic Generation of Vulnerability-Specific Patches for Preventing Component Hijacking Attacks in Android Applications. In *Proceedings of the 21th Annual Network and Distributed System Security Symposium (NDSS'14)*. San Diego, CA.